Lecture 10
Recovery and Under the Hood

The reflog, `reset`, `stash`, `bisect`, the `.git` folder, and where to go next

Learning goals

- Recover "lost" commits with the reflog, and undo a bad reset, merge or rebase.
- Explain the three modes of `git reset` and use each safely.
- Park unfinished work with `git stash`.
- Find the commit that introduced a bug with `git bisect`.
- Describe what is inside `.git`: blobs, trees, commits, refs, and why IDs behave as they do.
- Choose the right command for a situation, and know where to learn more.

1 Git rarely loses what you committed

By now you have used several commands that sound dangerous: `amend`, `rebase`, `force-with-lease`, `reset`. The good news of this last lecture is that, once something is *committed*, Git almost never really loses it. Commits that no branch points to any more stay in the repository for weeks. The skill is knowing how to find them. (Work that was *never* committed is different: `git restore` and `reset --hard` destroy it for good, as you learned in Lecture 3.)

We use a practice repository again, with no server this time. From the course folder:

```
$ bash codeHints/L10/setup_demo10.sh
$ cd practice/demo10
$ git log --oneline
6529ff1 Add waffle recipe
9d18d71 Add crepe recipe
28a460c Add pancake recipe
143b77c Add README
```

(It also contains the branches `side` and `chain`, used later. To rebuild it: `cd ../..` and run the script with `--reset`.)

2 The reflog: Git's flight recorder

Imagine you want to undo your last two commits and type a command too eagerly (`git reset` is explained properly in the next section; for now: "move `main` back two commits and make my files match"):

```
$ git reset --hard HEAD~2
HEAD is now at 28a460c Add pancake recipe
$ git log --oneline
28a460c Add pancake recipe
143b77c Add README
```

The crepe and waffle commits have vanished from the log. Are they gone? No. `main` just points to an older commit; the two commits are still stored, only nothing leads to them:

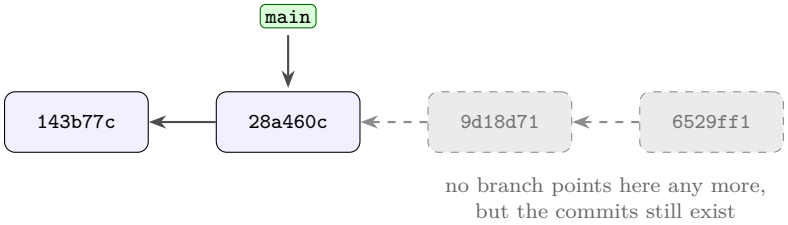


Figure 1: After `reset --hard HEAD~2`. The branch label moved back; the two newer commits (dashed) were not deleted.

Git keeps a private diary of where `HEAD` has been: the **reflog** (reference log). It records every commit, checkout, reset, merge and rebase *on your computer*:

```
$ git reflog
28a460c HEAD@{0}: reset: moving to HEAD~2
6529ff1 HEAD@{1}: checkout: moving from chain to main
7b5a37b HEAD@{2}: commit: Step 8
...
6529ff1 HEAD@{14}: commit: Add waffle recipe
9d18d71 HEAD@{15}: commit: Add crepe recipe
...
```

(The lines in between are the history of the demo setup. A long reflog opens in a pager: press `q` to leave it, as in Lecture 3.) Each line reads: *the commit `HEAD` was on after this action*, `HEAD@{n}` ("`n` moves ago"), and what happened. So the line `6529ff1 HEAD@{1}: checkout ...` is the position just *before* the reset, which is the state we want back. The numbers shift with every new action, so always look again instead of trusting a remembered number. Two ways to return:

```
$ git reset --hard HEAD@{1}              # move main back to where it was
HEAD is now at 6529ff1 Add waffle recipe

$ git branch rescue 6529ff1              # or: keep main as it is, label the lost
    commit
```

The second form is the safest: it creates a new branch `rescue` at the lost commit without touching anything else, and you can then inspect it, merge it, or delete it.

Key idea

After any mistake, *stop*, run `git reflog`, find the entry from just before the mistake, and put a branch on it (`git branch rescue <id>`). That works for deleted branches, bad resets, bad rebases and lost amend commits.

Three more cases, each with the same recipe:

- **A branch deleted with `-D`** (Lecture 4). Git prints the ID when it deletes: `Deleted branch side (was 85a4832)`. Bring it back with `git branch side 85a4832`. If you missed the line, `git reflog` still lists that commit.
- **An amend that went wrong** (Lecture 3). The reflog has a line such as `commit (amend): <new message>`; the line *below* it is the old commit. `git branch rescue <old-id>` saves it.
- **A rebase that went wrong** (Lecture 8). The reflog shows `rebase (start) ... rebase (finish)`; the entry just below `rebase (start)` is your branch before the rebase. Use that ID for `git reset --hard <id>` (or, if nothing else happened since, `git reset --hard ORIG_HEAD`).

Quick fact

The reflog is local: it is not pushed, and a fresh clone starts with a nearly empty one. (Our practice repositories switch off automatic clean-up, so their reflogs stay complete.) Its entries expire (by default after 90 days, or 30 for commits no branch reaches), after which `git gc` (Git's housekeeping command) may delete such *unreachable* commits, those that no branch or tag leads to, for good. That is why "find it soon" is the rule.

3 `git reset`: moving the branch

You met `reset --hard` above. `git reset <commit>` always does one thing first: it **moves the current branch** to that commit. The three *modes* differ in what happens to the staging area and your files (the three areas of Lecture 2):

	branch moves	staging area	working directory	Typical use
`--soft`	yes	kept	kept	redo the last commits (squash)
`--mixed` (default)	yes	reset	kept	unstage everything; recommit selectively
`--hard`	yes	reset	**reset**	throw everything away

Starting each time from `6529ff1`, here is `git reset --<mode> HEAD~1` and its effect on `git status --short`:

```
$ git reset --soft HEAD~1
$ git status --short
M  recipes.txt              # the waffle change is STAGED (M in the left column)
$ git reset --hard 6529ff1  # back to the waffle commit for the next try

$ git reset --mixed HEAD~1
$ git status --short
 M recipes.txt              # in your file, NOT staged (M in the right column)
$ git reset --hard 6529ff1

$ git reset --hard HEAD~1
$ git status --short        # nothing printed: the change is gone from your files too
$ git reset --hard 6529ff1  # bring the waffle commit back for the next examples
```

(`--mixed` also prints `Unstaged changes after reset:` before the status. After `--hard` the file has one line fewer; the commit itself is still in the reflog, which is why the last line works.)

Pitfall

`--hard` discards uncommitted work permanently. Committed work it only moves out of sight (the reflog has it); *uncommitted* changes are gone for good. Before you use it, run `git status` and `git diff`. And never reset commits that you have already pushed and shared: that rewrites history under your teammates' feet (the golden rule of Lecture 8). For shared mistakes use `git revert` (Lecture 3).

3.1 Three recipes

Squash the last commits (`--soft`). You made three "wip" commits and want one clean commit, without an interactive rebase:

```
$ git reset --soft HEAD~3
$ git commit -m "Add feature"
```

Undo a merge or rebase that you have not pushed (`ORIG_HEAD`). Before a risky operation (merge, rebase, reset, pull) Git sets a bookmark called `ORIG_HEAD` at where `HEAD` was. The next such operation overwrites it:

```
$ git merge --no-edit side
$ git log --oneline --graph -4
*   f7f7cbe Merge branch 'side'        (your merge ID differs)
|\
| * 85a4832 Add sale banner
* | 6529ff1 Add waffle recipe
|/
$ git reset --hard ORIG_HEAD       # I did not want that merge
HEAD is now at 6529ff1 Add waffle recipe
```

The same works right after a bad `git rebase`. (If something else happened in between, use the reflog instead.)

Remove a commit from the wrong branch. In Lecture 8 you copied a commit made on the wrong branch to the right one with `cherry-pick`. The copy on the wrong branch (if you have not pushed it) is removed with `git switch wrongbranch` and `git reset --hard HEAD~1`. Check with `git status` first that you have nothing uncommitted.

4 `git stash`: parking unfinished work

In Lecture 4 you saw that Git refuses to switch branches when uncommitted edits would be overwritten, and you were told that `stash` is the answer. A **stash** is a shelf where Git puts your uncommitted changes aside and gives you a clean working directory:

```
$ echo "half-done idea" >> recipes.txt
$ git status --short
 M recipes.txt
$ git stash push -m "idea for later"
Saved working directory and index state On main: idea for later
$ git status --short                # clean: nothing printed
$ git stash list
stash@{0}: On main: idea for later
$ git switch side                   # do the urgent work elsewhere ...
$ git switch main
$ git stash pop                     # take the shelf back
 M recipes.txt
```

(`pop` also prints the full status.) `git stash pop` re-applies the changes and removes the entry from the shelf. Undo the experiment with `git restore recipes.txt`.

Command	Effect
`git stash push -m "text"`	Shelve uncommitted changes of tracked files (`-u` also shelves new, untracked files).
`git stash list`	Show the shelf, newest first (`stash@{0}`, `stash@{1}`, ...).
`git stash pop`	Re-apply the newest entry and delete it.
`git stash apply`	Re-apply but keep the entry (useful to try on several branches).
`git stash drop`	Delete the newest entry without applying it.

Pitfall

A stash is easy to forget. For anything you will keep for more than a day, *commit it on a work-in-progress branch* instead. A stash is local, not backed up, and `pop` can conflict (resolve as in Lecture 5; the entry then stays until you drop it).

5 `git bisect`: finding the commit that broke it

"It worked last month, and now it is broken." Reading hundreds of commits is hopeless. **Bisect** finds the guilty commit by *binary search*: you tell Git one commit that is good and one that is bad; Git checks out the one in the middle; you say whether it is good or bad; Git halves the range again. Among 1000 commits this takes only about 10 tests.

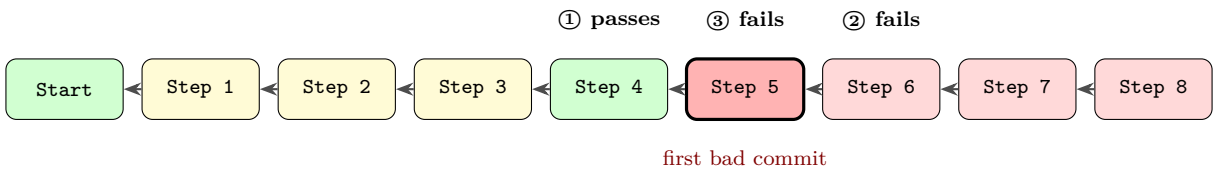
Our `chain` branch has a test script `test.sh` that fails once `status.txt` contains a line with the word `BROKEN`.

We know that the first commit (`Start chain`, `6150894`) is good and the newest (`Step 8`) is bad:

```
$ git switch chain
$ git bisect start
$ git bisect bad                 # the current commit (Step 8) is bad
$ git bisect good 6150894        # this old commit was good
Bisecting: 3 revisions left to test after this (roughly 2 steps)
[79e7d89...] Step 4
```

Git is now sitting on `Step 4` in the middle. You can test by hand and answer `git bisect good` or `git bisect bad`, or let Git run your test script for every step. A script that exits with 0 means "good", non-zero means "bad":

```
$ git bisect run bash test.sh
running  'bash' 'test.sh'
Bisecting: 1 revision left to test after this (roughly 1 step)
[1e6de4b...] Step 6
running  'bash' 'test.sh'
Bisecting: 0 revisions left to test after this (roughly 0 steps)
[29234e3...] Step 5
running  'bash' 'test.sh'
29234e3d2c3598e2072617999ff1aa980efeab52 is the first bad commit
    Step 5
$ git bisect reset               # IMPORTANT: leave bisect mode, back to your branch
```



green = good, red = bad, yellow = never tested: three tests were enough for the nine commits

Figure 2: The search of the demo: test ① `Step 4` (good), ② `Step 6` (bad), ③ `Step 5` (bad). Since `Step 4` is good and `Step 5` is bad, `Step 5` is the culprit.

Pitfall

Always finish with `git bisect reset`; until then you are on a temporary, detached state (Lecture 4). Bisect needs a *reliable* test (the same result every time), and a commit that cannot be tested at all can be skipped with `git bisect skip`.

6 Under the hood: what is inside `.git`?

You have used Git for ten lectures; now look inside. The hidden folder from Lecture 1 is a small database plus a few pointer files:

```
$ ls .git
COMMIT_EDITMSG  HEAD  ORIG_HEAD  config  description  hooks  index
info  logs  objects  refs
$ cat .git/HEAD
ref: refs/heads/main
$ cat .git/refs/heads/main
6529ff1c759ea9c51798a7e5852fd5f21964cb57
```

- `HEAD` is a one-line file: "I am on branch `main`" (Lecture 4's picture, literally).
- A **branch** is a file containing a commit ID: 40 characters, nothing else. That is why creating a branch costs nothing. (After a clone or a `git gc` you may also see a file `packed-refs`: Git's tidy summary of many such files, in which case the branch's file can be missing.)
- `index` is the staging area. `logs` is the reflog. `config` holds the local settings of Lecture 1. `hooks` can hold scripts that Git runs at certain moments.
- `objects` is the database of everything you ever committed.

6.1 Four kinds of objects

Everything in `objects` is one of four kinds, each stored under the **hash** of its contents:

Object	What it holds
blob	the contents of one file (no name, no folder)
tree	a folder: a list of names, each pointing to a blob (file) or another tree (subfolder)
commit	a pointer to one tree (the snapshot), the parent commit(s), author, date, message
tag	an annotated tag (Lecture 9): a name, a message, and a pointer to a commit

You can look at them with `git cat-file`:

```
$ git cat-file -t HEAD
commit
$ git cat-file -p HEAD
tree aeba1480b8783d748b9cd09f5ad0cbc64f127587
parent 9d18d71a8420fffe8667946d288bfd39d1ef5fd7
author Practice User <practice@example.com> 1738659600 +0000
committer Practice User <practice@example.com> 1738659600 +0000

Add waffle recipe
```

```
$ git cat-file -p HEAD^{tree}
100644 blob ae5e354863163528256fab6c692aaada622e83bd    README.md
100644 blob 010497540ae142ff657f07122465b4fc646c4bb1    recipes.txt
$ git cat-file -p HEAD:recipes.txt
Pancakes: flour, milk, eggs
Crepes: flour, milk, eggs, butter
Waffles: flour, milk, eggs, sugar
```

A commit is thus tiny: a tree ID, a parent ID, who and when. The "snapshot" of Lecture 2 is the tree it points to.

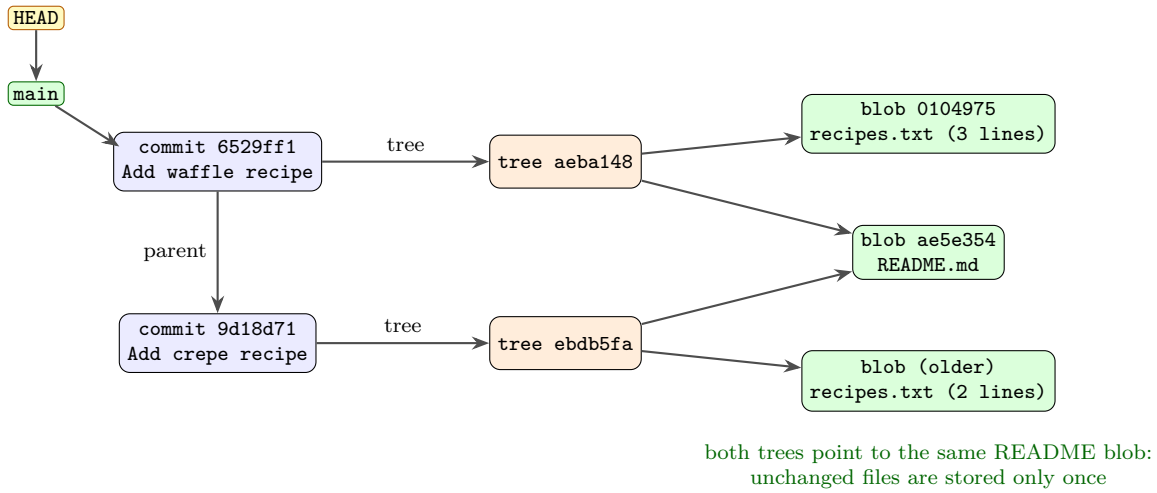


Figure 3: Two commits as stored objects. The README did not change, so both trees point to the very same blob.

6.2 Why IDs behave the way they do

The ID of an object is a **hash** (SHA-1, 40 hexadecimal characters) of its content. Same content, same ID, anywhere in the world:

```
$ echo "hello" | git hash-object --stdin
ce013625030ba8dba906f756967f9e9ca394464a
```

This one idea explains much of what you learned:

- *Unchanged files are stored once* (Figure 3): same content, same blob.
- *Rebase makes "copies" with new IDs* (Lecture 8): a commit contains its parent's ID, so a different parent means different content, hence a different hash.
- *History cannot be changed silently*: altering an old commit changes its ID and every ID after it.
- *Branches are cheap*: just a file with an ID in it.

Objects that nothing points to (the "lost" commits of the reflog section) are *unreachable*; `git gc` removes them after their reflog entries have expired. (Git also packs many loose objects into compact *pack files*; newer Git can even use SHA-256 instead of SHA-1: the ideas stay the same, only the IDs differ. You rarely need to think about either.)

7 Which command for which situation?

I want to…	Use (lecture)
see where I am and what changed	`git status`, `git diff`, `git log --oneline --graph` (2, 3, 4)
undo an unstaged or staged change	`git restore file`, `git restore --staged file` (3)
fix my last commit (unshared)	`git commit --amend` (3)
undo a shared commit safely	`git revert` (3)
work on an idea separately	`git switch -c name` (4)
combine branches	`git merge`; fix conflicts, `git add`, commit (5)
share, download	`push`, `fetch`, `pull`, `clone` (6)
propose a change	branch, push, pull request (7)
tidy my unshared commits	`git rebase -i`, `reset --soft` (8, 10)
copy one commit	`git cherry-pick` (8)
find a lost commit	`git reflog`, then `git branch rescue <id>` (10)
park unfinished work	`git stash push -m` / `pop` / `apply` (10)
undo a fresh merge, rebase or reset	`git reset --hard ORIG_HEAD` (10)
look at an old commit	`git switch --detach <id>` (4)
find the commit that broke it	`git bisect run` (10)
name a version	`git tag -a`, release (9)

8 Where to go from here

You can now do everything a working developer does with Git every day. To keep growing:

- **Practise on real projects.** Use Git for your own notes, papers or code from today on. Commit small, read `git status` before and after every command, make a branch for every idea.
- **Contribute.** Look for the label *good first issue*; `firstcontributions/first-contributions` on GitHub is a safe first pull request.
- **Read.** *Pro Git* (free at `https://git-scm.com/book`) goes deeper on every topic; `git help <command>` is always at hand.
- **Play.** `https://learngitbranching.js.org` shows branches and rebases as animated pictures. `https://ohshitgit.com` lists rescue recipes for common mistakes. `https://skills.github.com` offers short interactive GitHub courses.
- **Discover more tools:** `git blame` (who wrote this line?), `git log -S"text"` (when did this text appear?), aliases, hooks, submodules, Git LFS, signed commits.

Key idea

Four habits cover most of Git: **commit small and often; read `git status`; branch for every idea; never rewrite what others have**. If you break the last one by accident, `git reflog` is your friend.

9 Homework 10

Homework 10: Rescue, squash, bisect, shelve

The setup script builds `practice/hw10`: a repository where two commits were lost by a hard reset, a branch of messy commits, an urgent bug, and a chain of commits with a hidden bug. From the course folder:

```
$ bash codeHints/L10/setup_hw10.sh
$ cd practice/hw10
```

Do the tasks in order. To start over: `setup_hw10.sh --reset` from the course folder. Set your identity (`git config user.name` and `user.email`) if it is not set globally.

10.1 Rescue

On `main`, two commits including one called `Important work` were lost by a hard reset. Find it with `git reflog` (the reflog is long: press `q` to leave the pager, or filter it with `git reflog | grep Important`) and create a branch `rescued` that points at the commit named exactly `Important work` (`Start important work` is a different, older commit). Do *not* move `main`.

10.2 Squash

The branch `messy` has three commits (`wip`, `wip 2`, `more wip`) on top of `main`. Using `git reset --soft`, turn them into *one* commit called `Add feature` with exactly the same files.

10.3 Bisect

The branch `history` is a chain of 12 commits; a bug appeared at some step. Its first commit (`git rev-list --max-parents=0 history`) is good, its newest is bad, and `test.sh` in the repository tells good from bad. Use `git bisect run bash test.sh` to find the first bad commit, finish with `git bisect reset`, and write `bad-commit=<first 7 characters of its ID>` in `practice/hw10_answers.txt` (next to `hw10`; use `printf` or `echo`).

10.4 Stash

Switch to `main`. Append the line `work in progress` to `notes.txt` and *do not commit*. Now an urgent bug arrives: the branch `urgent` has the typo `teh` in `app.txt`. Shelve your change with `git stash`, switch to `urgent`, fix the typo, commit it as `Fix urgent bug`, switch back to `main`, and `git stash pop`. At the end `git status --short` must show only `M notes.txt` (a space, then `M`) and the stash must be empty.

10.5 Look inside

Add three more lines to the answers file: `tree=<first 7 characters of the ID of main's tree>` (use `git cat-file -p main`), `hello=<first 7 characters of the hash of the text hello>` (use `printf 'hello\n' | git hash-object --stdin`; run this in Git Bash, other shells may add different line endings), and `ref-file=<first 7 characters stored in .git/refs/heads/main>` (use `cat`; if the file does not exist, Git has packed it, so use `git rev-parse main` instead).

Check yourself: from the course folder run `bash codeHints/L10/check_hw10.sh`. All `PASS`, then compare with the solutions at the end of this lecture (or in `solutions/L10/`).

10 Summary and farewell

- Committed work is almost never lost: `git reflog` lists where `HEAD` has been; `git branch rescue <id>` saves a lost commit. `ORIG_HEAD` undoes a fresh merge, rebase or reset.
- `reset --soft/--mixed/--hard` move the branch and keep, unstage, or discard your changes. `--hard` destroys uncommitted work; never reset shared history.

- `git stash` shelves uncommitted work; `git bisect run` finds the commit that broke something by binary search.
- Inside `.git`: branches are files holding IDs, commits point to trees, trees to blobs, and every ID is a hash of content.

You have finished the course. In Lecture 0 we started with a folder full of `report_v2_final_REALLY_final.docx` files and asked for a time machine and a collaboration tool. You now have both: from `git init` to rebase, pull requests and recovery. Keep the four habits, keep practising, and enjoy using Git. (*Pro Git*, Chapters 7.3 and 10.)

Solutions

Stop here if you have not tried the homework yet.
Attempt every task first, run the checker, and only then read on.

Solutions to Homework 10

```
$ bash codeHints/L10/setup_hw10.sh
$ cd practice/hw10
$ git reflog | grep "Important work"            # 10.1
0b30b44 HEAD@{18}: commit: Important work      (your number differs)
$ git branch rescued 0b30b44                      # use the ID you see
$ git switch messy                                # 10.2
$ git reset --soft main
$ git commit -m "Add feature"
$ git switch history                              # 10.3
$ git bisect start
$ git bisect bad
$ git bisect good $(git rev-list --max-parents=0 history)
$ git bisect run bash test.sh
... is the first bad commit
$ git bisect reset
$ git switch main                                 # 10.4
$ echo "work in progress" >> notes.txt
$ git stash
$ git switch urgent
$ sed -i.bak 's/teh/the/' app.txt && rm app.txt.bak     # or edit by hand
$ git commit -am "Fix urgent bug"
$ git switch main
$ git stash pop
$ git status --short
 M notes.txt
```

For 10.5 (the three IDs are examples, yours differ except the hash of `hello`):

```
$ git cat-file -p main | head -1
tree 8cfff4b...
$ printf 'hello\n' | git hash-object --stdin
ce013625030ba8dba906f756967f9e9ca394464a
$ cat .git/refs/heads/main
1689350...
$ printf 'bad-commit=e238140\ntree=8cfff4b\nhello=ce01362\nref-file=1689350\n' > ../
    hw10_answers.txt
        # (writes all four answer lines at once: bad-commit from 10.3, the rest for
    10.5)
```

10.1. The reset moved `main` back, but the reflog remembers every position of `HEAD`. Your reflog is long because it also contains the history of the setup; `grep` finds the line. `git branch rescued <id>` only adds a label, so `main` stays where it was. (If you used `git reset --hard <id>` on `main`, the rescue works but `main` moved, and the checker complains.)

10.2. `git reset --soft main` moves `messy` back to `main`'s commit but keeps all the changes *staged*, so a single `git commit` makes one commit with exactly the same files as the three wip commits (the checker compares the file contents).

10.3. `bisect run` tests about $\log_2 12 \approx 4$ commits. The first bad commit is `Step 7`. Do not forget

`git bisect reset`: otherwise the repository stays in bisect mode and the checker reports it.

10.4. A stash lets you switch branches with a clean working directory. After the pop, your uncommitted line is back in `notes.txt`, and the commit `Fix urgent bug` lives on `urgent`, not on `main`. If `git stash pop` reports a conflict, you changed the same lines on both sides; resolve it as in Lecture 5.

10.5. `git cat-file -p main` prints a commit: its first line gives the ID of its tree. The hash of `hello` is the same on every computer (`ce01362...`). The file `.git/refs/heads/main` holds the full ID of `main`'s commit, so its first 7 characters equal `git rev-parse --short=7 main`. (If the file does not exist, Git has packed the reference into `.git/packed-refs`; use `git rev-parse main` instead.)

Common mistakes.

- Moving `main` to the lost commit in 10.1 instead of creating a branch.
- Forgetting `git bisect reset`; or running bisect while having uncommitted changes (stash or commit first).
- Committing the `work in progress` line instead of stashing it in 10.4: then `main` has a new commit and the check "only `M notes.txt`" fails.
- Writing fewer than 7 characters of an ID: write at least 7 (the full 40 are accepted too).